

哈尔滨工程大学 2023 年新生程序设计竞赛

A_Big_Jiong

哈尔滨工程大学 程序设计与算法学社

2023 年 10 月 19 日

目录

1 Easy

- A 哈尔滨工程大学
- B 气泡统一方程
- C 小 Z 的子串
- D 铁道，启动！
- E 保护隐私

2 Middle

- F 优美串
- G 折纸游戏
- H Rubyonly 杀

3 Hard

- I 信使国际象棋
- J 序列计数

Easy
Middle
Hard

A 哈尔滨工程大学
B 气泡统一方程
C 小 Z 的子串
D 铁道，启动！
E 保护隐私

A 哈尔滨工程大学

根据题意“哈尔滨工程大学源自 1953 年创办的中国人民解放军军事工程学院”可以知道，哈尔滨工程大学的创办年份为 1953 年，使用任何语言直接输出即可。

Easy
Middle
Hard

A 哈尔滨工程大学
B 气泡统一方程
C 小 Z 的子串
D 铁道，启动！
E 保护隐私

B 气泡统一方程

要求计算 $R^2\ddot{R} + 2R\dot{R}^2$ 的值，其中 $R(t) = at^2 + bt + c$ ，求导则有 $\dot{R} = 2at + b$, $\ddot{R} = 2a$ ，计算输出即可。

对于使用 C 或 C++ 的选手，由于 int 所能表示的数据范围有限，故需要使用 long long 类型（或更大），同时样例也提醒了这一点。

C 小 Z 的子串

根据题目中提供的定义，子串为字符串中连续的一段，那么 k 个 `heu` 子串至少需要 $3k$ 个字符，如果 $n < 3k$ ，显然不存在解，输出 `-1`。

对于存在解的方案，考虑先放置 $3k$ 个字符，即用循环输出 k 次 `heu`。对于剩余的 $n - 3k$ 个字符，可以简单地输出 $n - 3k$ 个 `h` 即可。

本题存在多种满足题意的构造方式，其他方式如果可以满足题意也可。

D 铁道, 启动!

好评如潮的题目 (题面), 简单来说就是求 n 的第 k 小因子。

一个很显然的做法是枚举 $i = 1, 2, \dots, n$, 如果说 $n \% i = 0$ (即 i 是 n 的因子), 那么将 i 记录到数组中, 因为枚举顺序从小到大, 那么数组内也是从小到大存储的, 输出数组中第 k 个数就好 (编号从 1 开始), 如果数组内数字个数小于 k 的话, 输出 -1 即可。

事实上, 本题只需要枚举到 $\sqrt{n}$ 即可, 通过一定方式也可以通过本题, 不过限于难度设置, 本题并没有强制要求枚举到 $\sqrt{n}$ 。

E 保护隐私

本题主要是对 ASCII 码相关知识的考察，对于具体的 ASCII 码相关的知识，可以从百度上自行学习。当然，原题面下面也提供了一个可以生成部分 ASCII 码的程序，方便使用 ASCII 码。

做法是非常简单的，我们只需要遍历（从头到尾依次扫描）一下字符串，对不同类型的字符进行计数，进而判断若干条条件是否满足即可。

F 优美串

总结一下题意，就是当相邻的两个字符不同时，可以删除左侧的字符，最后要求删为一个字符。那么我们可以根据相邻字符不同的性质，将字符串看作若干个 0 和 1 串交替形成的字符串，即 $00\dots0011\dots10\dots01\dots$ 。

我们先考虑判定，即解决什么样的字符串是优美的的问题。我们发现，对于一个不同字符交替的位置，其左侧的若干的相邻的字符可以全部被删除，即 $000001111\dots$ 可以变为 $1111\dots$ 。当然，上面的 0 和 1 互换也是可以的。

F 优美串

那么，对于上面提到的 $00\dots0011\dots10\dots01\dots$ ，我们可以考虑从每个不同字符交替的位置出发，从左往右考虑，然后借由这个端点删去左侧的字符，即 0000111001 先变为 111001 ，然后是 001 ，最后变为 1 进而满足题目要求。

借由上面的例子，我们发现，如果说最后一个 0 或 1 串的长度不为 1 ，我们最后就会剩余这个 0 或 1 串，从而不能够满足题目的要求。举个例子， 00001110011 最后就会剩余 11 从而不能满足题意。

F 优美串

最后我们考虑计数，我们考虑枚举子串的右端点 R （编号从 1 开始）。只要原字符串的第 R 个位置和第 $R - 1$ 个位置不相同，那么从最左侧到 R 的一共 R 个子串都是合法的，向答案中累计 R 。否则，只有第 R 个字符本身的子串是合法的，向答案中累计 1。然后最后累计得到的和，就是这字符串中优美的子串的个数。

另外，非常抱歉的是，本题由于出题时在 Windows 系统下进行的，没有考虑过 Windows 系统和 Linux 系统之间换行符的差异，并且由于校队的疏忽，没有反复对这道题进行验证，再次表示抱歉。请不要使用 `getchar()` 读入数据。

G 折纸游戏

本题如果不考虑纸的形状的问题，答案显然为 $n + 1$ ，那么我们就来考虑什么情况下纸的形状会重复，然后用 $n + 1$ 减去重复的数量即可。

习惯上，我们记较短的边为 x ，较长的边为 y ，如果 $\nexists z, x \times 2^z = y$ ，那么不管怎样折叠，都不会出现重复的形状，所以说答案为 $n + 1$ 。

G 折纸游戏

结论

对于重复的纸片形状，每个形状仅会出现两次。

证明：形状重复的产生源自于 $a \times a$ 形状的纸片，由于这种纸片折叠后会形成两个相同形状的纸片 $a \times \frac{a}{2}$ 和 $\frac{a}{2} \times a$ ，而按照不考虑重复的计算策略，我们会分别计算两种纸片产生的数量。但是，我们规定形状的定义后，他们便会同时产生相同的形状，因此形状会重复两次。

G 折纸游戏

有了上面的结论，我们便可以着手计算重复的数量，现在有 $\exists z, x \times 2^z = y$ ，我们可以先用 z 次折叠将纸片折为正方形，那对于剩余的 $n - z$ 步操作，我们可以生成 $n - z + 1$ 种纸片（先不考虑形状），由上面的结论我们可知，存在 $\lfloor \frac{n-z+1}{2} \rfloor$ 个相同的形状，答案应该为 $n + 1 - \lfloor \frac{n-z+1}{2} \rfloor$ 。

本题在出题的时候考虑难度，将时间复杂度为 $O(T \log n)$ 的解法放过，事实上本题可以对于每个询问 $O(1)$ 解答，共计 $O(T)$ ，但是需要注意 C++ 下 $\log$ 函数的精度问题。

H Rubyonly 杀

本题直接给出博弈方案：

- 首先，Rubyonly 首先使用完所有的斩，然后如果能够用完所有的杀杀死 ylnoybuR，Rubyonly 取得胜利，否则不会使用任何杀。
- 然后，ylnoybuR 会先使用完所有的斩，然后再使用所有的杀，如果能够杀死 Rubyonly，ylnoybuR 获胜。
- 否则，双方都无法杀死对面，平局。

赛前，Rubyonly 信誓旦旦的跟我说 Rubyonly 杀是一个 Hard 难度的题目，考虑到 Rubyonly 是高超的算法大师，这个题在难度 rk8 的位置待了很久，结果竟然是水题，这个题的实际难度排序应当向前调整为 rk6。

I 信使国际象棋

本题的题解实在是好写，根据题意模拟即可。如果将后的走法分解为车和信使，然后每次行棋的时候单独判断该棋子周围有无信使，以及目标位置上有无己方棋子、是否涉及到吃子可有效降低代码长度和编写难度。有验题人使用了 `class` 来编写程序，可增强代码可调试性（当然 `struct` 也可以实现此点）。

作为出题人，已经对自己在赛场上出大模拟的行为进行了深刻的检讨，下次还出（不是）。此外，对赛时成功通过此题的佩了个佩 `pei` 队伍表示祝贺和敬意！

J 序列计数

前言

本题涉及快速数论变换等多项式算法知识，对于初学者或算法基础较差的同学，此题可以暂时忽略。

题目的本质是在 A_i 的 m 个二进制位上，填入一些 1，使得 $\sum_{i=1}^n \text{cnt}_1(A_i \otimes A_{n-i+1}) = k$ 成立。

J 序列计数

性质 1

当 k 为奇数时，方案数为 0。

证明：当枚举到 i 和 $n - i + 1$ 时， cnt_1 的贡献必定会计算相等的两次，所以说 k 一定为偶数。

性质 2

如果我们限定 $cnt_1(A_i \otimes A_{n-i+1})$ 的贡献，那对于 A_i 的某一种方案， A_{n-i+1} 都可以找到一个对应的方案。换言之，我们可以将题意转化为在 $\lfloor \frac{n}{2} \rfloor$ 个位置上产生 w_i 的贡献，使得 $\sum_{i=1}^{\frac{n}{2}} w_i = \frac{k}{2}$ 。

J 序列计数

性质 3

若 n 为奇数，中间的位置上的数字是可以任意填的，也就是我们可以在性质 2 求得答案的基础上，乘上 2^m ，得到的就是最终答案。

记 $dp(i, j)$ 为前 i 个位置上，共计产生了 j 个 1 的方案数，则有：

$$dp(i, j) = \sum_{u=0}^{\min\{m, j\}} dp(i-1, j-u) \times \binom{m}{u} \times 2^u \times 2^{m-u}$$

后面的 2^u 表示这 u 个贡献位上， A_i 填 0 或 1 的总方案数 (A_{n-i+1} 相反)， 2^{m-u} 表示无贡献的 $m-u$ 个位置上， A_i 填 0 或 1 的总方案数 (A_{n-i+1} 相同)。

J 序列计数

我们发现，上面的式子对于 j 为卷积的形式，对于 i 则是多次连续卷积，使用多项式快速幂即可求解，时间复杂度 $O(k \log^3 k)$ ，如果不使用牛顿迭代写法的话，时间复杂度为 $O(k \log^2 k \log n)$ ，对于常数较小的写法，本题也予以通过。

另外有人提供了更加多项式化的思考角度，因为总和起来要恰好等于 k ，对于方案数是乘积，每个的 cnt_1 个数是相加，刚好对应多项式相乘的时候系数乘，指数相加，记 $P_i = \binom{m}{i} \times 2^i \times 2^{n-i}$ ，就有多项式 $\sum_i P_i x^i$ ，直接使用多项式快速幂即可。